

CMPUT 267: Machine Learning I · Fall 2026

Mini-Batch and Stochastic Gradient Descent

Xi Ye

2026-10-06

Assignment 4 Is Out

1. **Open:** Use the [Assignment 4 student notebook](#) in Google Colab.
2. **Copy:** Select **File** → **Save a copy in Drive** and work in your own copy.
3. **Learn:** If Colab is new to you, review the [Google Colab introduction](#).
4. **Complete:** Run the setup cells, follow the notebook instructions, and fill in only the solution blocks marked `### YOUR CODE HERE ###`. Keep the markers and provided code unchanged.
5. **Submit:** Restart the runtime and run all cells. Download the `.ipynb` file, then upload it to **Assignment 4 on Canvas**.

Recommended completion: October 20, 11:59 p.m. MT

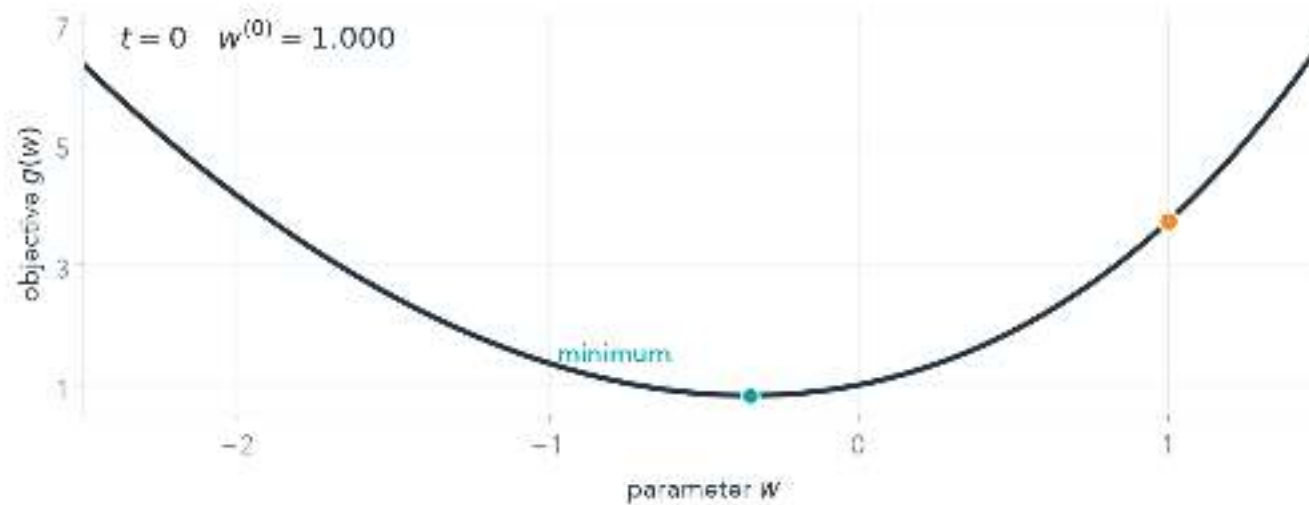


Hard deadline: October 27, 11:59 p.m. MT

A complete submission receives full assignment credit. Test results provide diagnostic feedback.

Recall: Gradient Descent

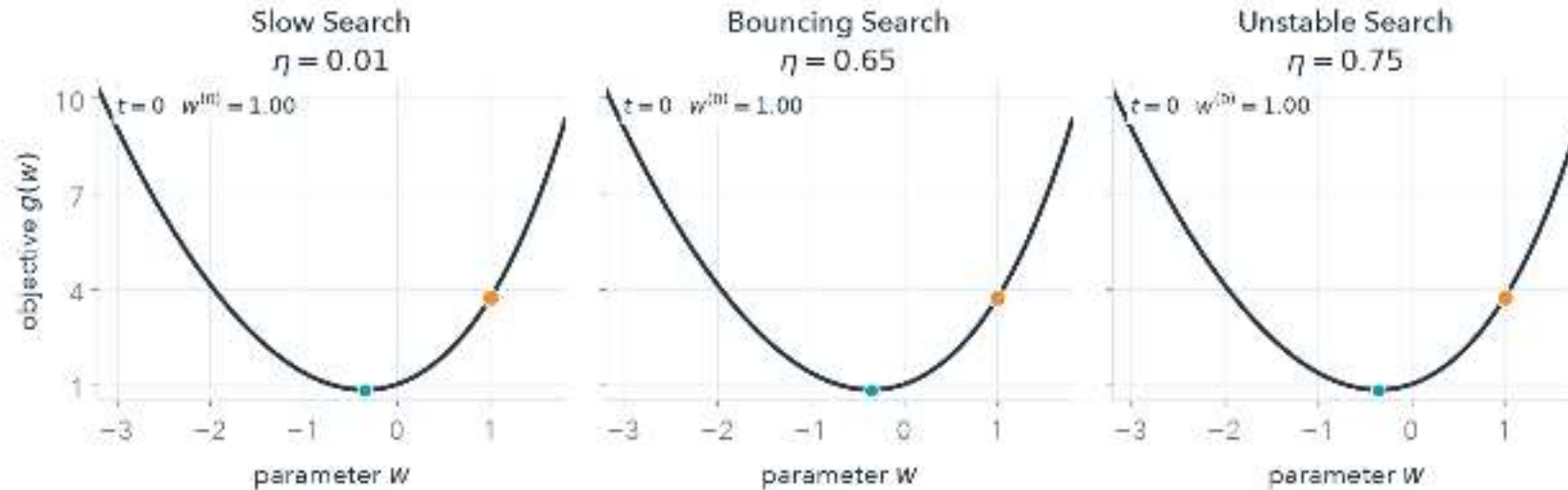
At the current parameters, use the gradient to choose a downhill direction.



$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta^{(t)} \nabla g(\mathbf{w}^{(t)})$$

Learning Rate and Scheduling

The same objective and initialization can produce very different searches.



i We often decrease the learning rate during training: larger steps early, then smaller steps near a minimizer. Following schedules like exponential decay.

Batch Gradient Descent for Linear Regression

For squared-loss linear regression, the full-data gradient is

$$\nabla \hat{L}_{\mathcal{D}}(\mathbf{w}) = \frac{2}{n} \sum_{i=1}^n (\mathbf{x}_i^{\top} \mathbf{w} - y_i) \mathbf{x}_i.$$

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta^{(t)} \nabla \hat{L}_{\mathcal{D}}(\mathbf{w}^{(t)}).$$



Batch gradient descent averages all n example gradients before each parameter update.

The Cost of a Batch Update

$(\mathbf{x}_1, y_1)$

$(\mathbf{x}_2, y_2)$

$(\mathbf{x}_3, y_3)$

$\dots$

$\dots$

$(\mathbf{x}_{n-2}, y_{n-2})$

$(\mathbf{x}_{n-1}, y_{n-1})$

$(\mathbf{x}_n, y_n)$

$n \text{ examples} \times O(d) \text{ work per example} = O(nd) \text{ per update.}$

For a large dataset, waiting for one batch update can be expensive.

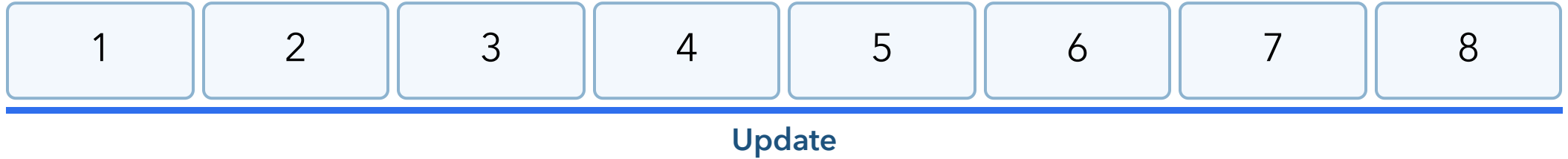
❓ Must every parameter update pass through all n examples?

Batch and Mini-Batch Updates

For the same eight examples, smaller groups update the parameters sooner.

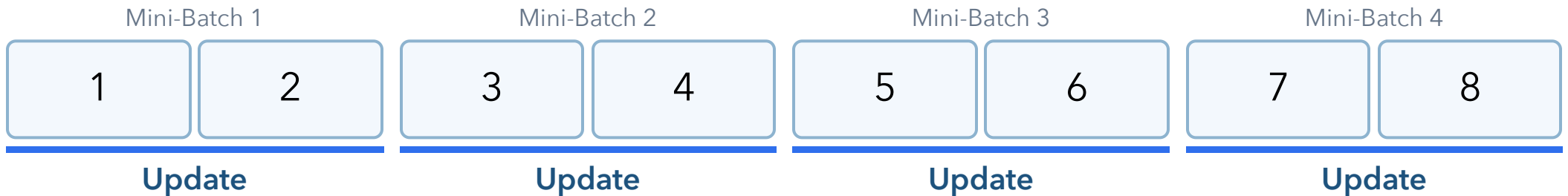
Batch GD

$b = 8$



Mini-Batch GD

$b = 2$



Batch Size and Update Cost

Batch Size b

The number of examples used to compute one gradient estimate.

$$\nabla \hat{L}_B(\mathbf{w}) = \frac{1}{b} \sum_{i \in B} \nabla L_i(\mathbf{w}), \quad |B| = b.$$

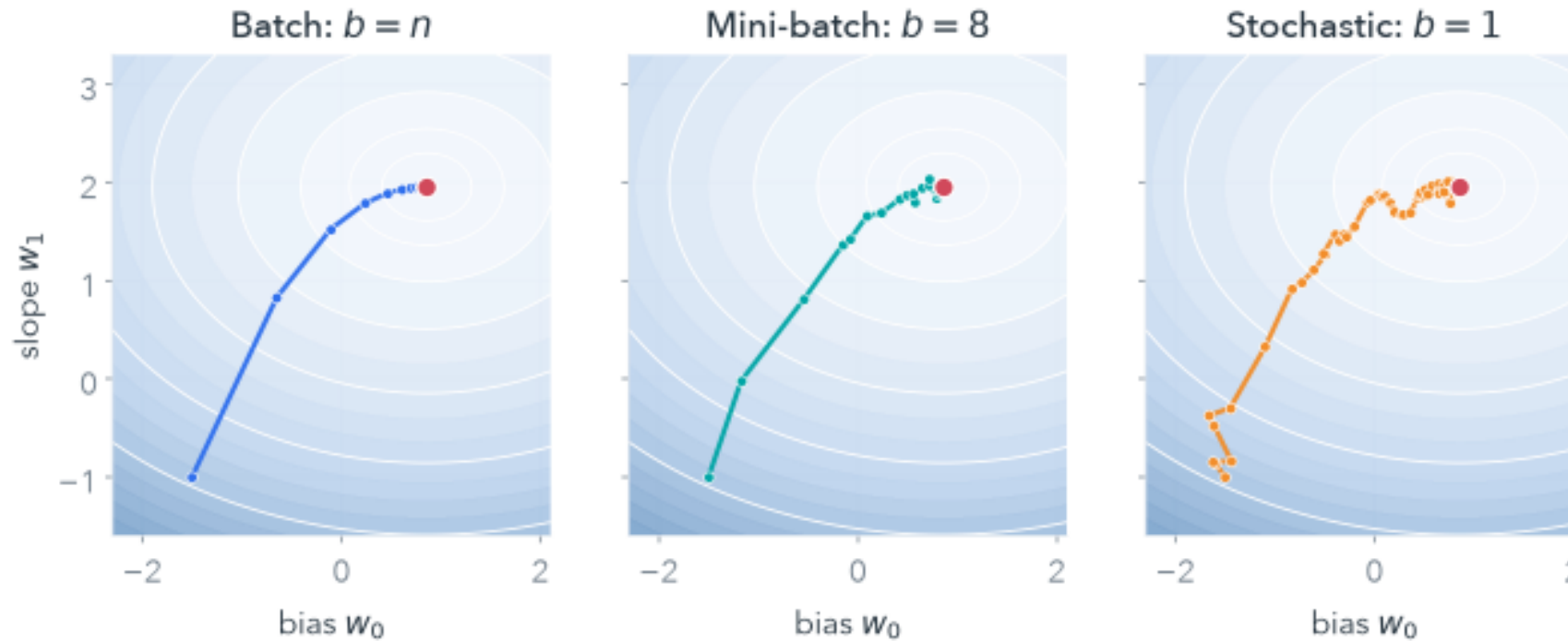
cost per update = $O(bd)$.

$b = n$ gives **batch gradient descent**

① $1 < b < n$ gives **mini-batch gradient descent**

$b = 1$ gives **stochastic gradient descent**.

Batch Size Changes the Trajectory



i All three paths optimize the same linear-regression objective. Smaller batches update more often, but their directions vary more.

Recall: Empirical Risk Estimates Population Risk

Fix $\mathbf{w}$ before drawing the dataset, and let $Z = \ell(f_{\mathbf{w}}(X), Y)$.

$$L(\mathbf{w}) = \mathbb{E}[Z], \quad \hat{L}_{\mathcal{D}}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\mathbf{w}}(\mathbf{x}_i), y_i).$$

$$\mathbb{E}_{\mathcal{D}}[\hat{L}_{\mathcal{D}}(\mathbf{w})] = L(\mathbf{w}), \quad \text{Var}_{\mathcal{D}}[\hat{L}_{\mathcal{D}}(\mathbf{w})] = \frac{\sigma_{\ell}^2(\mathbf{w})}{n}.$$



The empirical risk has the correct expected value. More data reduces its variance at rate $1/n$.

A Mini-Batch Estimates Empirical Risk

Now hold $\mathcal{D}$ fixed. Sample B , a set of b indices, uniformly without replacement.

$$\hat{L}_B(\mathbf{w}) = \frac{1}{b} \sum_{i \in B} \ell(f_{\mathbf{w}}(\mathbf{x}_i), y_i).$$

$$\mathbb{E}_B \left[\hat{L}_B(\mathbf{w}) \mid \mathcal{D} \right] = \hat{L}_{\mathcal{D}}(\mathbf{w}), \quad \text{Var}_B \left[\hat{L}_B(\mathbf{w}) \mid \mathcal{D} \right] = \frac{n-b}{n-1} \frac{\sigma_{\ell, \mathcal{D}}^2(\mathbf{w})}{b}.$$

(Here $\sigma_{\ell, \mathcal{D}}^2(\mathbf{w})$ is the variance of the n per-example losses in $\mathcal{D}$.)

① Conditioned on $\mathcal{D}$, the mean is the full empirical risk. The variance decreases as b increases.

① Larger batch size reduces the variances of estimate per mini batch.

The Mini-Batch Update

At mini-batch update k :

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta^{(k)} \nabla \hat{L}_{B_m}(\mathbf{w}^{(k)})$$

- i Same update rules. The mini-batch gradient replaces the full-data gradient.

Mini-Batch Gradient Descent

Input: dataset $\mathcal{D}$, learning-rate schedule, **epoch** T (number of passes over entire dataset), batch size b

1. Initialize $\mathbf{w}$.
2. For each pass $t = 1, \dots, T$:
 - **shuffle the training examples.**
 - form $M = \text{floor}(n/b)$ full mini-batches and discard the remainder.
 - for $m = 1, \dots, M$:
 - compute $\mathbf{g} \leftarrow \nabla \hat{L}_{B_m}(\mathbf{w})$.
 - update $\mathbf{w} \leftarrow \mathbf{w} - \eta \mathbf{g}$.
3. Return $\hat{f}(\mathbf{x}) = f_{\mathbf{w}}(\mathbf{x})$.

Epochs and Number of Updates

Epoch

One pass through the shuffled dataset. We train for T epochs and discard the incomplete final batch.

b

Batch size: examples used for one update

$M = \text{floor}(n/b)$

Updates in one epoch

TM

Total updates after T epochs



Shuffle at the start of every epoch so that the order of the data points is random for each epoch.

Why Shuffle Every Epoch?

Without shuffling, **the same examples batch repeatedly appear together and in the same order.**

Fixed Order

The i -th batch of an epoch will always contain the same group of samples.

The group and the order affects the learning.

The learning process is biased by the order.

Shuffled Each Epoch

New groups provide varied gradient estimates across passes through the data.

 Shuffling reduces artifacts from the stored order.

Two Ways to Handle the Remainder

Suppose $n = 10$ and $b = 4$.



Discard the Remainder

$$M = \text{floor}(10/4) = 2$$

Use B_1 and B_2 . Examples 9 and 10 are unused in this epoch.

Use a Smaller Final Mini-Batch

$$B_3 = \{9, 10\}$$

Make a third update and average its gradient using $|B_3| = 2$.

Cost per Update and per Epoch

For d parameters:

Method	Batch size	Updates per epoch	Cost per update	Cost per epoch
Batch GD	n	1	$O(nd)$	$O(nd)$
Mini-batch GD	b	$\text{floor}(n/b)$	$O(bd)$	$O(nd)$
Stochastic GD	1	n	$O(d)$	$O(nd)$

All three process about n examples per epoch.

- i The cost per epoch is the same. But smaller batch size induces more frequent updates per epoch.

Equal Epochs Do Not Mean Equal Paths

After one epoch, every method has processed about the same number of examples.

$$\begin{aligned} b = n &: 1 \text{ update,} \\ b = 32 &: \text{floor}(n/32) \text{ updates,} \\ b = 1 &: n \text{ updates.} \end{aligned}$$

i More frequent updates can make faster early progress, but noisy directions can also slow or destabilize optimization.

Typicall, tere is no universally best batch size.

Batch Size and Learning Rate Interact

The learning rate cannot be chosen independently of the gradient variability.

Larger batches

- More stable gradient estimates
- May tolerate a larger learning rate
- Require more memory per update

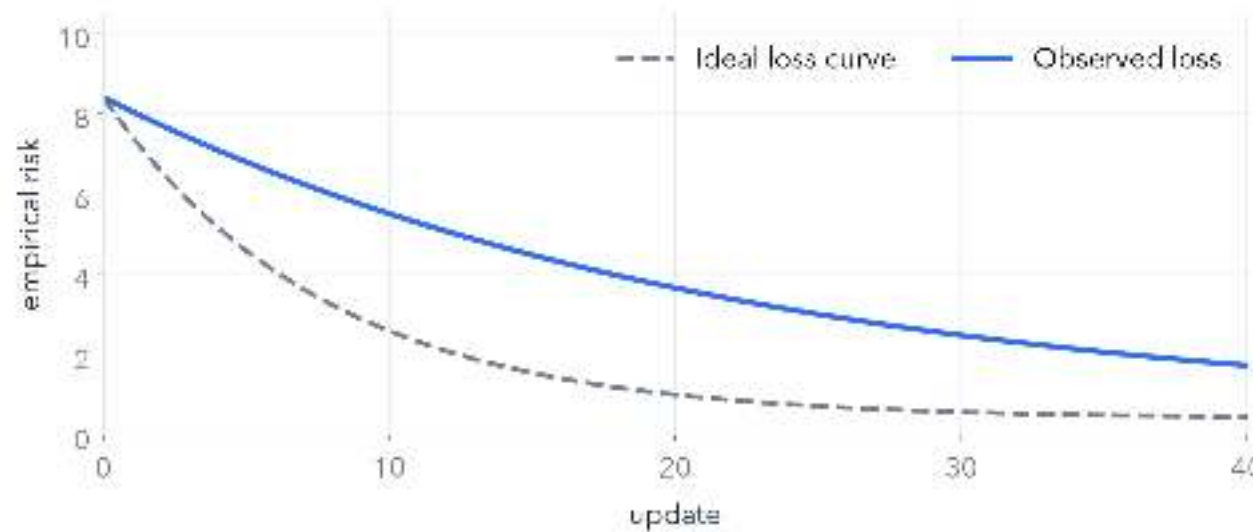
Smaller batches

- Noisier gradient estimates
- May require a smaller learning rate
- Provide more updates per data pass

 Batch size and learning-rate schedule are hyperparameters selected together.

Reading Loss Curves; we typically monitor training using loss curves (loss vs training steps)

Slow, Steady Decrease



Observed behavior

Loss decreases consistently, but much more slowly than the ideal trend.

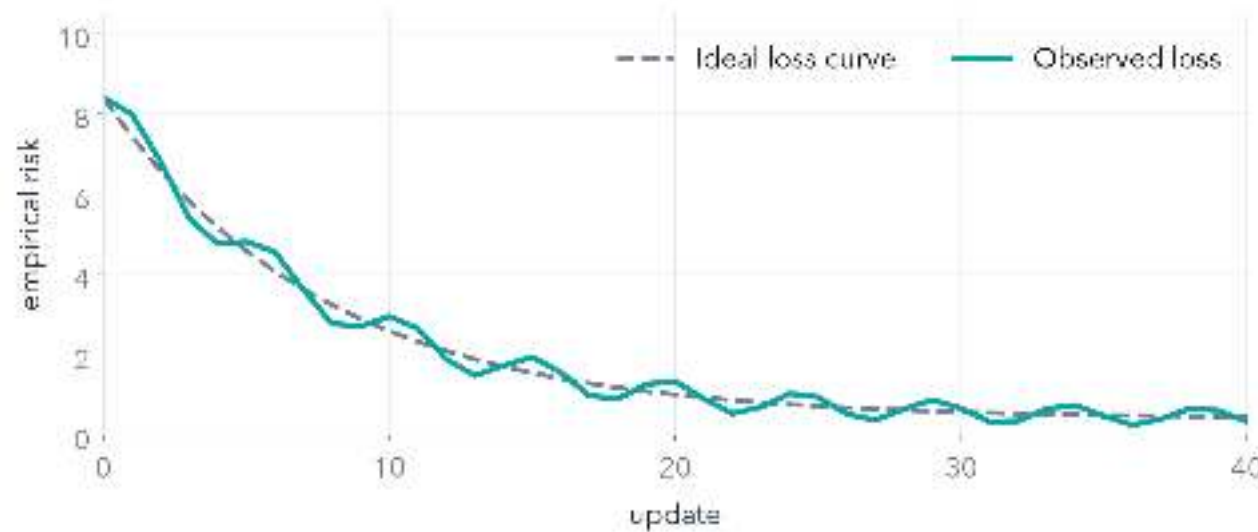
Likely interpretation

The learning rate may be too conservative.

First response

Try a moderately larger initial learning rate and check that training remains stable.

Noisy but Improving



Observed behavior

Individual updates fluctuate around a clear downward trend.

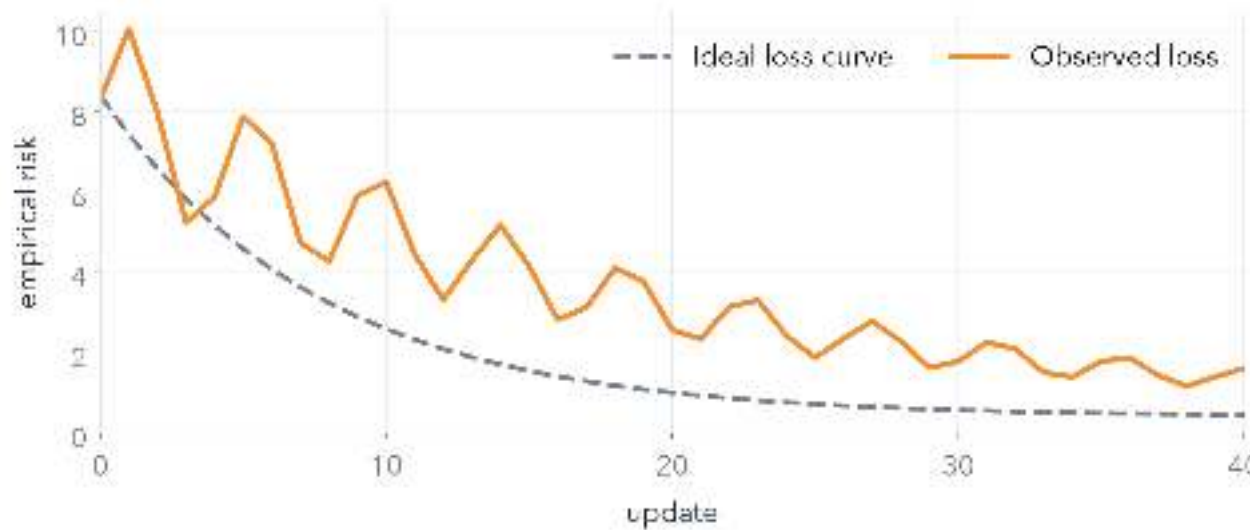
Likely interpretation

This is often ordinary mini-batch variation, especially when b is small.

First response

Track a moving average or the full-data loss before changing the optimizer.

Large Oscillations



Observed behavior

Loss repeatedly overshoots and swings above and below the ideal trend.

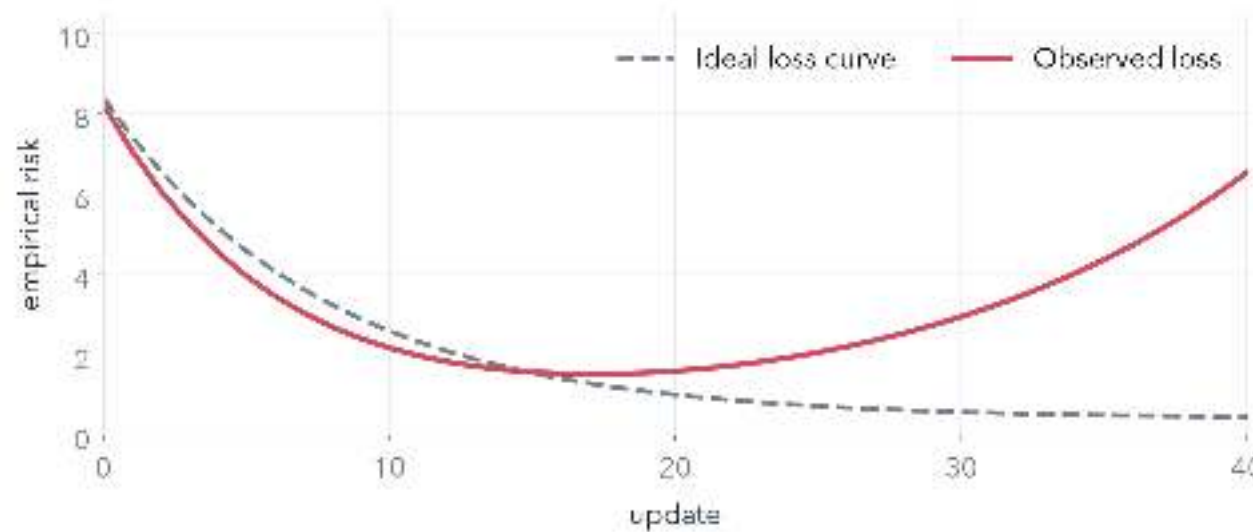
Likely interpretation

Updates may be too aggressive for the variability of the gradient estimates.

First response

Reduce the learning rate. If needed, also increase the batch size.

Sustained Increase



Observed behavior

Loss increases for many consecutive updates after brief initial progress.

Likely interpretation

Optimization is diverging, usually because the learning rate is too large.

First response

Stop the run and reduce the learning rate before restarting.

Closed Form and Iterative Optimization

For squared-loss linear regression with n examples and d parameters:

Normal Equations

$$O(nd^2 + d^3)$$

One exact linear-system solution under the assumptions from September 22.

Batch Gradient Descent

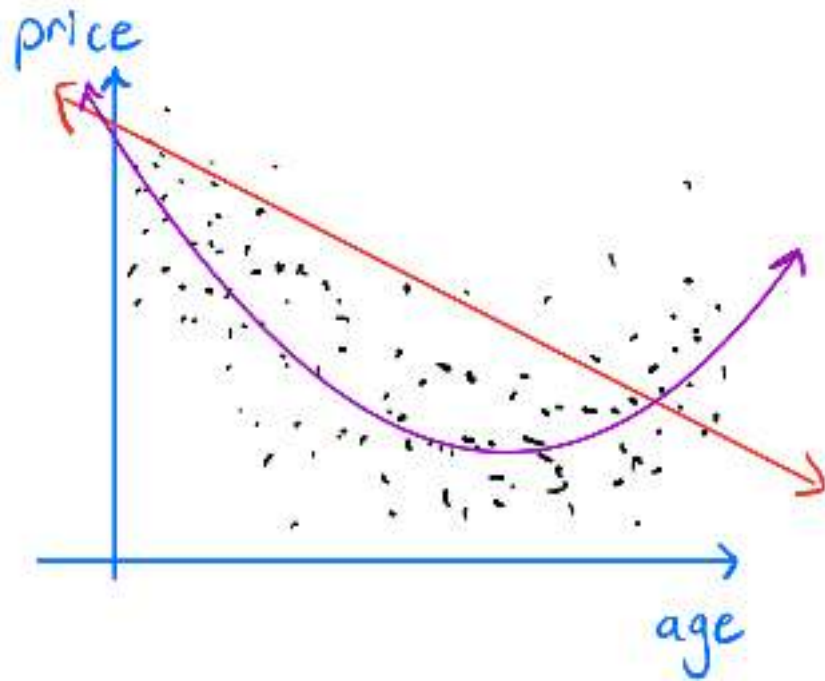
$$O(Tnd)$$

T approximate full-data updates without forming a matrix inverse.

 These expressions describe broad scaling for the algorithms presented here. Practical solvers and hardware can change the comparison.

Polynomial Regression

Why Linear Functions Can Fail



Suppose we predict a car's price from its age.

The price may fall quickly, level off, and later rise when the car becomes an antique.

No line can capture this curved relationship.

i Changing the optimizer cannot fix a function class that contains only lines.

One Feature: Polynomial Feature Maps

A **feature map** transforms the original input into a new feature vector.

$$\phi_2(x) = \begin{pmatrix} 1 \\ x \\ x^2 \end{pmatrix} \quad \text{and} \quad \phi_3(x) = \begin{pmatrix} 1 \\ x \\ x^2 \\ x^3 \end{pmatrix}.$$

We call ϕ_2 and ϕ_3 the **degree-two** and **degree-three polynomial feature maps**.

transformed using ϕ_2 : $f_{\mathbf{w}}(x) = w_0 + w_1x + w_2x^2$,

transformed using ϕ_3 : $f_{\mathbf{w}}(x) = w_0 + w_1x + w_2x^2 + w_3x^3$.

 Increasing the degree adds another power of x and another parameter. The predictor remains linear in $\mathbf{w}$.

Two Features: Squares and Interactions

For $\mathbf{x} = (x_1, x_2)$, the degree-two map contains every product with total degree at most two:

$$\phi_2(\mathbf{x}) = (1 \quad x_1 \quad x_2 \quad x_1^2 \quad x_1x_2 \quad x_2^2)^\top.$$

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2.$$

 The product x_1x_2 is an **interaction**. If x_1 is width and x_2 is length, then x_1x_2 is area. The model can use this combined effect even when the original inputs contain only width and length.

Multivariable Polynomial Feature Maps

Let $\mathbf{x} = (x_1, \dots, x_d)^\top$ and choose a maximum degree p . Each polynomial feature is a **monomial**, a product of powers of the inputs.

$$\mathcal{M}_{d,p} = \left\{ x_1^{a_1} \cdots x_d^{a_d} \mid (a_1, \dots, a_d) \in \mathbb{N}_0^d, \sum_{j=1}^d a_j \leq p \right\}.$$

Stack these monomials into the feature map:

$$\phi_p(\mathbf{x}) = (m_1 \quad \cdots \quad m_{\bar{p}})^\top, \quad \{m_1, \dots, m_{\bar{p}}\} = \mathcal{M}_{d,p}, \quad \bar{p} = \binom{d+p}{p}.$$

 The choice $a_1 = \dots = a_d = 0$ gives the bias feature 1. Products such as $x_1 x_2$ give interaction features.

Three Features: A Short Example

For $\mathbf{x} = (x_1, x_2, x_3)$ and degree two:

$$\phi_2(\mathbf{x}) = (1, x_1, x_2, x_3, x_1^2, x_1x_2, x_1x_3, x_2^2, x_2x_3, x_3^2)^\top.$$

$$d = 3, \quad p = 2, \quad \bar{p} = \binom{d+p}{p} = \binom{5}{2} = 10.$$

i Three measured features become ten polynomial features, including the bias term.

Polynomial Function Classes

For one input x , let $\mathcal{F}_p$ contain all polynomial predictors of degree at most p .

$$\mathcal{F}_1 = \{w_0 + w_1x \mid (w_0, w_1) \in \mathbb{R}^2\}, \quad \text{for example } 2 + 3x,$$

$$\mathcal{F}_2 = \{w_0 + w_1x + w_2x^2 \mid \mathbf{w} \in \mathbb{R}^3\}, \quad \text{for example } 1 - x + x^2,$$

$$\mathcal{F}_3 = \{w_0 + w_1x + w_2x^2 + w_3x^3 \mid \mathbf{w} \in \mathbb{R}^4\}, \quad \text{for example } x^3 - 2x.$$

 A lower-degree polynomial also belongs to every higher-degree class. Set the extra coefficients to zero.

Nested Classes and Training Loss

Allowing a higher degree gives a larger set of candidate predictors.

$$\mathcal{F}_1 \subseteq \mathcal{F}_2 \subseteq \mathcal{F}_3 \subseteq \dots$$

$$\min_{f \in \mathcal{F}_1} \hat{L}_{\mathcal{D}}(f) \geq \min_{f \in \mathcal{F}_2} \hat{L}_{\mathcal{D}}(f) \geq \min_{f \in \mathcal{F}_3} \hat{L}_{\mathcal{D}}(f) \geq \dots$$

Proof sketch. Let $\hat{f}_{p-1}$ minimize the loss over $\mathcal{F}_{p-1}$. Because $\hat{f}_{p-1} \in \mathcal{F}_p$, it is also a candidate at degree p . Therefore,

①

$$\min_{f \in \mathcal{F}_p} \hat{L}_{\mathcal{D}}(f) \leq \hat{L}_{\mathcal{D}}(\hat{f}_{p-1}) = \min_{f \in \mathcal{F}_{p-1}} \hat{L}_{\mathcal{D}}(f).$$

How Do We Fit This Predictor?

For two features and degree two, the predictor has six parameters:

$$f_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_5x_2^2.$$

❓ Do we need a new optimization method for polynomial regression?

Answer: Linear Regression After Feature Mapping

Transform every input first:

$$\mathbf{z}_i = \phi_2(\mathbf{x}_i) = \begin{pmatrix} 1 & x_{i1} & x_{i2} & x_{i1}^2 & x_{i1}x_{i2} & x_{i2}^2 \end{pmatrix}^\top.$$

$$f_{\mathbf{w}}(\mathbf{x}_i) = \mathbf{w}^\top \phi_2(\mathbf{x}_i) = \mathbf{w}^\top \mathbf{z}_i, \quad \min_{\mathbf{w}} \frac{1}{n} \sum_{i=1}^n (\mathbf{w}^\top \mathbf{z}_i - y_i)^2.$$

i **After feature mapping**, this is an ordinary linear regression problem with six transformed features.

Polynomial Regression Reuses Linear Regression

After transforming each input, polynomial regression is linear regression in $\phi_p(\mathbf{x})$:

$$\hat{L}_p(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n (\phi_p(\mathbf{x}_i)^\top \mathbf{w} - y_i)^2.$$

$$\nabla \hat{L}_p(\mathbf{w}) = \frac{2}{n} \sum_{i=1}^n (\phi_p(\mathbf{x}_i)^\top \mathbf{w} - y_i) \phi_p(\mathbf{x}_i).$$

 Replace $\mathbf{x}_i$ with $\phi_p(\mathbf{x}_i)$. The closed-form solution, batch GD, mini-batch GD, and SGD all apply.

Wrap-up

- Batch gradient descent uses all n examples for one update.
- Mini-batch gradient descent uses subsets of size b and updates several times per epoch.
- Stochastic gradient descent is the case $b = 1$.
- Smaller batches reduce work per update but increase gradient variability.
- Polynomial feature maps expand the function class with powers and interactions.
- Linear-regression optimizers apply after replacing $\mathbf{x}$ with $\phi_p(\mathbf{x})$.